PROCÉDURE · MON INFRASTRUCTURE HAUTE DISPONIBILITÉ · TESTS ET DÉPANNAGE

Plan de tests et matrice de validation

Plan de tests et matrice de validation : bascule des serveurs web, panne d'un nœud Galera, répartiteur et réseau.

> 🗂️ Document de mon dossier E6 (BTS SIO SISR), tel que je l'ai rédigé pendant la mise en place de mon laboratoire. Version du 06/11/2026. Les adresses sont celles de mon laboratoire : adapte-les au tien.

Tests de bascule, du cluster, du répartiteur et du réseau, avec leur état.

> 🔐 Aucun mot de passe ni clé dans ce document : les secrets sont remplacés par des marque-places.
>
> ⚠️ **Écarts avec le labo réel** (configuration relevée le 01/10/2026, voir le DAT §7.2) : HAProxy contrôle ses serveurs **toutes les 2 s** (`inter 2s fall 3 rise 2`), le contrôle SQL est un simple test de port (`tcp-check` , pas `mysql-check`), et il n'y a **ni page de statistiques (8404) ni en-tête `X-Served-By`** : les passages qui les citent décrivent une cible, pas l'existant. adm-01 est le contrôleur de domaine Windows : les commandes notées « sur adm-01 » se jouent depuis une session du bastion Guacamole.

Plan de tests

Objectif

Démontrer, par des tests **reproductibles** et **tracés**, que l'infrastructure atteint ses objectifs (DAT-Infrastructure.md) : fonctionnement nominal, tolérance aux pannes, filtrage.

Organisation

Famille	Tests	Document	Outils
Réseau	T01, T02, T18, T19	Matrice-de-flux.md	`check-network.sh` , `nc` , `curl`
Base de données	T03 à T08, T20	galera-tests.md	`check-galera.sh` , `test-galera-node.sh`
Répartition	T09, T10, T13	haproxy-tests.md	`check-haproxy.sh` , `test-sql-round-robin.sh`
Web	T11, T12, T14 à T16	wordpress-failover.md	`check-wordpress.sh` , `test-web0X-failure.sh`
GLPI	T17	haproxy-tests.md	`check-glpi.sh`

Synthèse et statuts : validation-matrix.md

Ordre d'exécution

```
T01-T02 (réseau, SSH)
   └─► T03-T08 (MariaDB, Galera)
         └─► T09-T10 (HAProxy SQL)
               └─► T11-T13 (WordPress, HAProxy HTTP)
                     └─► T17 (GLPI) ─► T18-T19 (NAT, firewall)
                           └─► T14-T16, T20 (pannes : en dernier, sur une base saine)
```

Les tests de panne ne sont lancés que lorsque **tous** les tests nominaux sont réussis : sinon, impossible de savoir si un échec vient de la panne simulée ou d'un défaut préexistant.

Règles

1. **Une preuve par test** : sortie de commande (fichier texte) ou capture d'écran, rangée dans evidence/.
2. **Statut honnête** : un test non rejoué depuis la dernière modification repasse en « À rejouer ».
3. **Date et version** : chaque campagne note la date et le commit du dépôt.
4. **Pannes réversibles** : les pannes sont simulées par arrêt de service (`systemctl stop`), jamais par suppression de données.

5. **Snapshot préalable** : avant une campagne de pannes, snapshot Proxmox des VM concernées.

Enregistrer une campagne

```
# SUR adm-01 — exemple d'archivage d'une sortie
./scripts/validation/check-wordpress.sh | tee "evidence/test-results/$(date +%F)-T13-haproxy-http.txt"

# Les scripts de failover ont une option dédiée
./scripts/failover/test-web01-failure.sh --save
```

Pour supprimer les codes couleur dans les fichiers : `NO_COLOR=1 ./scripts/...`

Critères de réussite globaux

Critère	Seuil
Tests nominaux (T01 à T13, T17 à T19)	100 % réussis
Tests de panne (T14 à T16, T20)	100 % réussis, 0 erreur client pendant la panne web
Délai de détection d'une panne web	≤ 10 s (`fall 3` × `inter 3s`)
Délai de réintégration	≤ 10 s (`rise 2` × `inter 3s`)

Matrice de validation

Légende des statuts

Statut	Signification
✅ Validé	Test exécuté, résultat conforme, **preuve archivée** dans `evidence/`
🟡 Déclaré	Fonctionnalité réalisée dans le lab (roadmap cochée), **preuve à produire et archiver**
⬜ À exécuter	Test jamais exécuté ou à rejouer depuis la dernière modification
❌ Échec	Test exécuté, résultat non conforme — incident ouvert
🔴 Bloqué	Test impossible tant qu'un prérequis manque (T17 : GLPI pas installé)

> Les statuts ci-dessous sont **initialisés d'après l'avancement déclaré du projet** (roadmap du README). Aucun test n'est marqué ✅ tant que sa preuve n'est pas présente dans evidence/. Mettre à jour cette page à chaque campagne.

Campagne : `AAAA-MM-JJ` — commit : `<COMMIT>` — opérateur : `<OPERATEUR>`

Matrice

ID	Composant	Test	Commande	Résultat attendu	Statut	Preuve
T01	Réseau	Passerelles et hôtes joignables	adm-01 : `scripts/validation/check-network.sh`	4 passerelles, 8 hôtes, ports autorisés ouverts	🟡 Déclaré	`02-pfsense-interfaces-vlan.png` , `*-T01-network.txt`
T02	SSH	Connexion par clé sur chaque VM	adm-01 : `ssh -o BatchMode=yes mysql01 hostname`	Connexion sans mot de passe ; mot de passe refusé	⬜ À exécuter	`*-T02-ssh.txt`
T03	MariaDB	Service actif, écoute sur l'IP vBDD	bdd-0X : `ss -ltnp | grep 3306`	`10.0.20.X:3306` sur les 3 nœuds	🟡 Déclaré	`*-T04-T07-mysql0X.txt`
T04	Galera	Taille du cluster	bdd-0X : `check-galera.sh`	`wsrep_cluster_size = 3`	✅ Passé le 30/10	`06-galera-3-noeuds-synced.png`

ID	Composant	Test	Commande	Résultat attendu	Statut	Preuve
T05	Galera	Composant primaire	bdd-0X : `check-galera.sh`	`wsrep_cluster_status = Primary`	✅ Passé le 30/10	`06-galera-3-noeuds-synced.png`
T06	Galera	Nœud synchronisé	bdd-0X : `check-galera.sh`	`wsrep_local_state_comment = Synced`	✅ Passé le 30/10 (3 nœuds)	`06-galera-3-noeuds-synced.png`
T07	Galera	Nœud prêt	bdd-0X : `check-galera.sh`	`wsrep_ready = ON`	✅ Passé le 30/10	`06-galera-3-noeuds-synced.png`
T08	Galera	Réplication multi-maître	INSERT sur chaque nœud, SELECT sur les autres (procédure)	Données identiques sur les 3 nœuds	🟡 Déclaré	`*-T08-replication.txt`
T09	HAProxy SQL	Connexion SQL via HAProxy	web-01 : `mariadb -h 10.0.10.2 -u wordpress -p -e 'SELECT @@hostname'`	Réponse d'un nœud Galera	🟡 Déclaré	`*-T09-sql.txt`
T10	HAProxy SQL	Répartition round-robin	haproxy : `test-sql-round-robin.sh -n 9`	3 nœuds distincts, 0 erreur	⬜ À exécuter	`*-T10-sql-round-robin.txt`
T11	WordPress	web-01 répond	adm-01 : `curl -I http://10.0.10.1`	HTTP 200/30x	🟡 Déclaré	`06-wordpress-web01.png`
T12	WordPress	web-02 répond	adm-01 : `curl -I http://10.0.10.4`	HTTP 200/30x	🟡 Déclaré	`07-wordpress-web02.png`
T13	HAProxy HTTP	Répartition HTTP	adm-01 : `check-wordpress.sh`	HAProxy UP, alternance `X-Served-By`	🟡 web-01 et web-02 UP (30/10) ; en-tête `X-Served-By` pas encore ajouté	`07-haproxy-etat-serveurs.png`
T14	Failover	Panne web-01	web-01 : `sudo systemctl stop apache2` ; adm-01 : `curl -I http://10.0.10.2`	0 erreur, `X-Served-By: web-02`	✅ Passé le 30/10 : web-01 DOWN en 9 s, 6 réponses sur 6	`08-test-panne-web01.png`
T15	Failover	Panne web-02	web-02 : `sudo systemctl stop apache2` ; adm-01 : `curl -I http://10.0.10.2`	0 erreur, `X-Served-By: web-01`	✅ Passé le 30/10 : web-02 DOWN en 9 s, 6 réponses sur 6	`09-test-panne-web02.png`
T16	Failover	Réintégration d'un backend	`sudo systemctl start apache2` ; adm-01 : `curl -I http://10.0.10.2`	Alternance rétablie en ≤ 10 s, sans action sur HAProxy	⬜ À exécuter	`*-failover-web01.log`
T17	GLPI	GLPI via HAProxy	adm-01 : `check-glpi.sh`	UP direct et via `:8080`	🔴 30/10 : GLPI affiche son assistant d'installation (base vide)	`11-glpi-ticket-incident-1.png` après installation
T18	NAT	Publication WAN	Poste labo : `curl -I http://10.4.0.198:8081`	HTTP 200/30x, `X-Served-By` présent	🟡 Déclaré	`*-T18-nat.txt`
T19	Firewall	Flux interdits bloqués	vUSER : `nc -zv -w 3 10.0.20.1 3306` (et liste)	Timeout sur chaque flux interdit	⬜ À exécuter	`02-pfsense-firewall-vweb.png` , `*-T19-firewall.txt`
T20	Galera	Perte d'un nœud	haproxy : `test-galera-node.sh --node 3`	`size = 2` , `Primary` , SQL OK ; retour `size = 3`	⬜ À exécuter	`*-T20-galera-node.txt`

ID	Composant	Test	Commande	Résultat attendu	Statut	Preuve
T21	Supervision	Zabbix toujours publié	Poste admin : navigateur `http://10.4.0.198:2231/zabbix/`	Page de connexion Zabbix	🟡 `/zabbix/` répond `200` en local le 06/11 ; à rejouer depuis le poste	`*-T21-zabbix.png`
T22	NAT	SSH de zabbix toujours publié	Poste admin : `Test-NetConnection 10.4.0.198 -Port 2232`	`TcpTestSucceeded : True`	⬜ À exécuter	`*-T22-ssh-zabbix.txt`
T23	NAT	LogAnalyzer joignable depuis le poste	Poste `10.2.0.122` : Chrome `http://10.4.0.198:2233/`	Demande d'authentification, puis interface	🟡 Passé le 06/11 (après correction du `403`, INC-05)	`16-loganalyzer-403-initial.png`, `22-syslog-recette-8-vm.png`
T24	Apache	Authentification LogAnalyzer	zabbix : `curl -I http://127.0.0.1:8085/` puis avec `-u <COMPTE_LOGANALYZER>`	`401`, puis `200` ; `/zabbix/` toujours `200`	🟡 Passé le 06/11 (sortie de commande)	`*-T24-curl.txt`
T25	LogAnalyzer	Assistant et source des journaux	Poste : `/install.php` (MO-9 §3)	Source `SYSLOG-CENTRAL`, *Diskfile*, `central.log`	🟡 Passé le 06/11 (LogAnalyzer 5.0.2)	`18-…`, `19-loganalyzer-source-diskfile.png`
T26	Rsyslog	Réception locale TCP et UDP	zabbix : `logger` local vers le port 514, `sudo tail central.log`	Les 2 messages de test (07:35:37, 07:35:42)	🟡 Passé le 06/11	`20-loganalyzer-tests-locaux.png`
T27	Rsyslog	Envois **directs** depuis les 8 VM Linux	chaque VM : `logger -n 10.0.30.3 -P 514 -T -t TEST-<VM> "…"`	8 lignes dans `central.log`	🟡 Passé le 06/11	`21-syslog-envois-directs.png`
T28	Rsyslog	Envoi **automatique** depuis les 8 VM Linux	chaque VM : `logger -t SYSLOG-AUTO "TEST AUTOMATIQUE DEPUIS $(hostname)"`	8/8 lignes `SYSLOG-AUTO` dans LogAnalyzer	🟡 Passé le 06/11 : **8/8**, plus des événements `sudo` et `mariadbd` réels	`22-syslog-recette-8-vm.png`
T29	Rsyslog	Événements de pfSense	fw-01 : *Remote Logging* vers `10.0.30.3:514`, puis une connexion à l'interface	Hôte pfSense visible dans LogAnalyzer	⬜ À exécuter	`*-T29-syslog-pfsense.png`
T30	Firewall	Source non autorisée bloquée	cli-01 : `Test-NetConnection 10.0.30.3 -Port 514`	`False` ; refus visible dans *Status / System Logs / Firewall*	⬜ À exécuter	`*-T30-syslog-bloque.txt`
T31	Firewall	`DEFAULT-DENY` dernière sur chaque interface	pfSense : *Firewall / Rules*, 5 onglets (après nouvel export)	Dernière ligne = `BLOCK ALL` partout ; PASS 514 au-dessus des isolements	⬜ À exécuter	`24-pfsense-regles-syslog.png`
T32	LogAnalyzer	Verrouillage après l'assistant	zabbix : `curl -I -u <COMPTE_LOGANALYZER> http://127.0.0.1:8085/install.php`	`404` ; messages toujours visibles	⬜ À exécuter	`*-T32-verrouillage.txt`
T33	Rsyslog	Résilience d'un émetteur	VM pilote : redémarrage, puis `logger -t SYSLOG-AUTO` ; coupure puis retour du 514	Messages reçus après redémarrage et après la coupure (file d'attente)	⬜ À exécuter	`*-T33-resilience.txt`
T34	Rsyslog	Rotation de `central.log`	zabbix : `logrotate -d`, puis rotation réelle en maintenance	Nouveau fichier `0640 root:www-data`, réception continue	⬜ À exécuter	`*-T34-logrotate.txt`

ID	Composant	Test	Commande	Résultat attendu	Statut	Preuve
T35	Rsyslog	Événements Windows	adm-01 et cli-01 (méthode à choisir)	Un événement Windows réel dans LogAnalyzer	À exécuter	`*-T35-windows.png`

Synthèse

Statut	Nombre
✅ Validé	6 (T04 à T07, T14, T15)
🟡 Déclaré (passé, preuve à archiver)	15
⬜ À exécuter	13
🔴 Bloqué	1 (T17)
❌ Échec	0

Recompté le 06/11/2026 sur 35 tests. T21 à T35 couvrent la journalisation centralisée (MO-9, registre du 06/11) : T23 à T28 passent à ✅ dès que les captures 16 à 22 sont déposées.

Objectif avant soutenance : **35** ✅, chaque preuve rangée selon la convention de `evidence/README.md` (dépôt GitHub).

Tests Galera

T03 — MariaDB opérationnel

SUR bdd-01, bdd-02, bdd-03

```
systemctl is-active mariadb                    # active
sudo ss -ltnp | grep 3306                      # écoute sur 10.0.20.X:3306 (pas 127.0.0.1)
```

T04 à T07 — État du cluster

SUR bdd-01, bdd-02, bdd-03

```
NO_COLOR=1 ./scripts/validation/check-galera.sh | tee "evidence/test-results/$(date +%F)-T04-T07-$(hostname -s).txt"
```

Test	Variable	Attendu
T04	`wsrep_cluster_size`	`3`
T05	`wsrep_cluster_status`	`Primary`
T06	`wsrep_local_state_comment`	`Synced`
T07	`wsrep_ready`	`ON`

Captures : `03-galera-cluster-size.png`, `04-galera-synced.png`

T08 — Réplication

Procédure : MO-1-Cluster-Galera.md

Critère : une ligne insérée sur un nœud est lisible immédiatement sur les deux autres, dans les trois sens.

T20 — Perte d'un nœud

Préparation

- T03 à T10 réussis.
- Snapshot Proxmox des 3 nœuds (recommandé).

- WordPress ouvert dans un navigateur (pour observer l'absence d'impact).

Exécution guidée

SUR haproxy

```
./scripts/failover/test-galera-node.sh --node 3
```

Le script indique la commande à lancer :

SUR bdd-03

```
sudo systemctl stop mariadb
```

Vérifications pendant la panne

Vérification	Attendu
`wsrep_cluster_size` sur bdd-01 et bdd-02	`2`
`wsrep_cluster_status`	`Primary` (2/3 = majorité)
SQL via HAProxy (6 connexions)	6 succès, bdd-03 absent des réponses
Page de stats HAProxy	bdd-03 **DOWN**
WordPress	Fonctionnel

Retour

SUR bdd-03

```
sudo systemctl start mariadb         # PAS galera_new_cluster
```

Vérification	Attendu
`wsrep_cluster_size`	`3` sur les 3 nœuds
bdd-03	`Synced` (après IST)
HAProxy	bdd-03 **UP**

Variante : écritures pendant la panne

Insérer des lignes pendant que bdd-03 est arrêté (via bdd-01), puis vérifier après son retour qu'il les possède : démontre l'**IST** (transfert incrémental).

Test complémentaire (non numéroté) — perte de deux nœuds

À réaliser **uniquement** en démonstration pédagogique, avec snapshot préalable :

1. Arrêt **propre** de bdd-02 puis bdd-03 : bdd-01 reste `Primary` avec `cluster_size = 1` (les départs ont été annoncés).
2. Pour illustrer le quorum, il faut une disparition **non annoncée** (ex. coupure réseau de deux nœuds via Proxmox, *Hardware → Network Device → Disconnect*) : le nœud restant passe `non-Primary` et refuse les requêtes.

Reprise : MO-1-Cluster-Galera.md.

Tests HAProxy

État général

SUR haproxy

```
NO_COLOR=1 ./scripts/validation/check-haproxy.sh | tee "evidence/test-results/$(date +%F)-haproxy-status.txt"
```

Attendu : service actif, syntaxe valide, ports 80/8080/3306/8404 en écoute, tous les serveurs `UP`.

Capture : page `http://10.0.10.2:8404/stats` depuis adm-01 → `05-haproxy-status.png`

T09 — HAProxy SQL

SUR web-01

```
mariadb -h 10.0.10.2 -P 3306 -u wordpress -p -e 'SELECT @@hostname, @@wsrep_node_name;'
```

Attendu : connexion réussie, nom d'un nœud Galera.

T10 — Round-robin SQL

SUR haproxy

```
NO_COLOR=1 ./scripts/utilities/test-sql-round-robin.sh -n 9 | tee "evidence/test-results/$(date +%F)-T10-sql-round-robin.txt"
```

Attendu : 9/9 connexions réussies, **3 nœuds distincts**.

T13 — HAProxy HTTP

SUR adm-01

```
for i in $(seq 1 6); do curl -s -o /dev/null -D - http://10.0.10.2/ | grep -Ei '^HTTP/|x-served-by'; done
```

Attendu : codes `200` (ou `30x`), `X-Served-By` alternant **web-01** / **web-02**.

Test complémentaire — health-check applicatif

Vérifie qu'un serveur dont l'**application** est cassée est retiré, alors qu'Apache tourne toujours (un check TCP ne verrait rien). On simule la panne en rendant `/wp-login.php` introuvable (réponse `404`, hors de la plage `200-399` attendue).

SUR web-02

```
sudo mv /var/www/wordpress/wp-login.php /var/www/wordpress/wp-login.php.test-hc
```

SUR adm-01 (après ~10 s)

```
for i in 1 2 3 4; do curl -sI http://10.0.10.2/ | grep -i x-served-by; done   # uniquement web-01
```

SUR web-02 (remise en état **immédiate**)

```
sudo mv /var/www/wordpress/wp-login.php.test-hc /var/www/wordpress/wp-login.php
```

Ce test prouve l'intérêt de `option httpchk` sur `/wp-login.php` par rapport à un simple check TCP.

T17 — GLPI via HAProxy

SUR adm-01

```
NO_COLOR=1 ./scripts/validation/check-glpi.sh | tee "evidence/test-results/$(date +%F)-T17-glpi.txt"
```

Attendu : glpi UP en direct et via `10.0.10.2:8080`, page identifiée comme GLPI.

Test de failover WordPress

Tests **T14** (panne web-01), **T15** (panne web-02) et **T16** (réintégration).

Objectif

Démontrer que l'arrêt d'un serveur web **n'interrompt pas** le site : HAProxy détecte la panne et envoie tout le trafic vers le serveur restant, puis réintègre automatiquement le serveur à son retour.

Prérequis

- [ ] T11, T12, T13 réussis (les deux serveurs répondent, alternance via HAProxy).
- [ ] Accès SSH à web-01 et web-02 depuis adm-01.
- [ ] Page de statistiques HAProxy ouverte (`http://10.0.10.2:8404/stats`) pour la démonstration visuelle.

Observation continue (recommandée)

SUR adm-01 — terminal dédié, laissé ouvert pendant tout le test :

```
while true; do
  printf '%s  ' "$(date +%T)"
  curl -s -o /dev/null -D - --max-time 3 http://10.0.10.2 | grep -Ei '^HTTP/|x-served-by' | tr -d '\r' | tr '\n' ' '
  echo
  sleep 1
done
```

T14 — web-01 arrêté : web-02 doit continuer

1. État initial

SUR adm-01

```
curl -I http://10.0.10.2
```

Plusieurs requêtes : `X-Served-By` alterne `web-01` / `web-02`.

2. Panne

SUR web-01

```
sudo systemctl stop apache2
```

3. Vérification

SUR adm-01

```
curl -I http://10.0.10.2
```

Critère	Attendu
Code HTTP	`200` (ou `30x`) — **jamais** `503` ni erreur de connexion
`X-Served-By`	`web-02` uniquement
Page de stats	web-01 **DOWN** (rouge) après ~9 s

Capture : `08-failover-web01-down.png`

4. Retour (T16)

SUR web-01

```
sudo systemctl start apache2
```

SUR adm-01 — après ~6 s :

```
curl -I http://10.0.10.2
```

`X-Served-By` alterne à nouveau `web-01` / `web-02` : **réintégration automatique**, sans action sur HAProxy.

T15 — web-02 arrêté : web-01 doit continuer

Procédure identique, en inversant les rôles.

SUR web-02

```
sudo systemctl stop apache2
```

SUR adm-01

```
curl -I http://10.0.10.2          # X-Served-By: web-01 uniquement
```

Capture : `09-failover-web02-down.png`

SUR web-02

```
sudo systemctl start apache2
```

Version scriptée

Les scripts reproduisent exactement cette procédure (mode guidé : ils indiquent la commande à lancer et attendent la confirmation de l'opérateur) :

```
# SUR adm-01
./scripts/failover/test-web01-failure.sh --save     # T14 + T16
./scripts/failover/test-web02-failure.sh --save     # T15 + T16
```

Option `--auto` : le script arrête et redémarre Apache lui-même via SSH (alias `web01` / `web02` de `~/.ssh/config`), après confirmation, et relance Apache s'il est interrompu.

Interprétation

Observation	Explication
Aucune erreur, même dans les premières secondes	Requêtes vers web-01 refusées puis **rejouées** sur web-02 (`retries 3` + `option redispatch`)
Quelques erreurs dans les premières secondes	Possible si le serveur ne refuse pas mais **ne répond plus** (VM figée) : les requêtes attendent le timeout jusqu'à détection (`fall 3`)

Observation	Explication
`503` permanent	Les deux serveurs sont DOWN : vérifier web-02 et le health-check

Variante : arrêt de la VM

Simuler une panne matérielle : **SUR L'HÔTE PROXMOX**, *Shutdown* (ou *Stop*) de la VM web-01. Le comportement attendu est le même ; la détection repose alors sur le timeout de connexion.
